Chapter 17 — Project Roadmap

Blueprints for building enterprise-grade AI applications from scratch to production. Each project lists architecture, build order, evaluation, and the chapters it exercises.

17.1 Reference Architecture (Capstone Skeleton)

Conceptual Deep Dive

```
Client -> API Gateway (auth, rate limit) -> App Service (FastAPI)
  -> Guardrails(in) -> Orchestrator (workflow/agent)
      -> Retriever (hybrid: dense+BM25 -> RRF -> reranker) -> Vector DB / Search
      -> Tools (SQL, APIs) with scoped credentials
      -> LLM Gateway (routing, fallback, cache) -> Provider / vLLM
  -> Guardrails(out) -> Response (stream via SSE)
Side planes: OTel traces + metrics, eval service, prompt registry, feature flags, queue + workers for ing
estion.
```

Build order for any project: (1) define metrics and golden set, (2) naive baseline, (3) instrument tracing, (4) improve with evals driving every change, (5) harden (reliability, security), (6) deploy with CI gates and canary, (7) operate (dashboards, feedback loop).

Cross-Connection Scenarios

Chapters 4, 7, 9, 10, 13, 14, 15 all appear.

Production Anti-Patterns

- Building the fancy orchestrator before a baseline and eval set exist.
- Skipping the ingestion pipeline's incremental updates and deletions.
- Treating "deployment" as the last step instead of designing for it up front.

Minimal Working Code

```
from dataclasses import dataclass, field
from typing import Callable

@dataclass
class Pipeline:
    guard_in: Callable[[str], bool]
    retrieve: Callable[[str], list[dict]]
    generate: Callable[[str, list[dict]], str]
    guard_out: Callable[[str], bool]
    trace: list = field(default_factory=list)

    def __call__(self, q: str) -> str:
        if not self.guard_in(q):
            return "Request blocked."
        docs = self.retrieve(q)
        self.trace.append(("retrieve", len(docs)))
        out = self.generate(q, docs)
        return out if self.guard_out(out) else "Response withheld."
```

17.2 Project A — Enterprise Knowledge Assistant (RAG)

Conceptual Deep Dive

Goal: answer employee questions over policies, wikis, tickets with citations and permissions. Components: connectors (Confluence, SharePoint, Drive) with incremental sync and deletion propagation; parsing and structure-aware chunking; metadata with ACLs; hybrid retrieval + reranker; permission-filtered queries; citation-enforcing prompts; abstention; feedback UI; eval set of 200 real questions with labelled sources; dashboards for recall, faithfulness, unanswered-question clusters. Milestones: baseline in a week, eval-driven tuning for three weeks, hardening and launch after that. Stretch: query routing between structured (SQL) and unstructured sources; agentic multi-hop for complex questions.

Cross-Connection Scenarios

Chapters 8, 9, 10, 14 (ACL), 13 (deploy).

Production Anti-Patterns

- Indexing documents the asker may not see.
- Never deleting removed documents from the index.
- Launching without a way to report bad answers.

Minimal Working Code

```
def sync(source_docs: dict[str, dict], index: dict[str, dict]):
    """source_docs/index map doc_id -> {hash, acl}. Returns (to_upsert, to_delete).&quo
t;""
    upsert = [d for d, m in source_docs.items() if d not in index or index[d]["hash"] != m[&quo
t;hash"]]
    delete = [d for d in index if d not in source_docs]
    return upsert, delete

print(sync({"a": {"hash": "1", "acl": ["hr"]}, "b&
quot;: {"hash": "2", "acl": []}},
          {"a": {"hash": "0", "acl": ["hr"]}, "c&
quot;: {"hash": "9", "acl": []}}))
```

17.3 Project B — Customer Support Copilot with Tools

Conceptual Deep Dive

Goal: draft replies and perform actions (look up order, issue refund within limits). Components: intent classifier (cheap model), RAG over help center, tool layer (order API, refund API with idempotency keys and amount caps), human-in-the-loop approval for refunds above threshold, tone/style via fine-tune or few-shot, PII redaction before logging, and CSAT/handle-time dashboards. Eval: replay historical tickets, judge draft quality and policy compliance, measure agent edit distance. Rollout: shadow mode (suggest only) → assisted → autonomous for low-risk intents.

Cross-Connection Scenarios

Chapters 6, 7, 4.5, 11, 14, 16.

Production Anti-Patterns

- Autonomous refunds with no cap or audit.
- Training on past agent replies that contain policy violations.
- Measuring only deflection and ignoring repeat contacts.

Minimal Working Code

```
def issue_refund(order_id: str, amount: float, idem_key: str, approved
: bool, limit=50.0,
                 done: dict | None = None):
    done = done if done is not None else {}
    if idem_key in done:
        return done[idem_key]
    if amount > limit and not approved:
        return {"status": "needs_approval"}
    done[idem_key] = {"status": "refunded", "order": order_id, "amount
": amount}
    return done[idem_key]

store = {}
print(issue_refund("o1", 20, "k1", False, done=store), issue_refund("o1", 2
0, "k1", False, done=store))
print(issue_refund("o2", 500, "k2", False))
```

17.4 Project C — Research / Coding Agent with Evaluation Harness

Conceptual Deep Dive

Goal: an agent that completes multi-step tasks (research a topic into a cited report; fix failing tests in a repo). Components: planner/executor graph (LangGraph) with checkpointing; tools (search, fetch, file edit, test runner in a sandbox); verifier (tests pass, citations resolve); budgets (steps, cost, time); trajectory logging; replayable evaluation harness running tasks N times and reporting pass^k, cost and step counts; human approval before commits. Stretch: multi-agent split (researcher, writer, critic) and memory for repeated users.

Cross-Connection Scenarios

Chapters 11 (incl. 11.6 sandboxing), 7, 10, 14, 15.

Production Anti-Patterns

- Agent edits run directly on the host.
- Evaluating by reading three transcripts.
- No spending cap per task.

Minimal Working Code

```
import statistics

def run_suite(agent, tasks, k=3):
    rows = []
    for t in tasks:
        runs = [agent(t) for _ in range(k)]          # each -> {"ok":bool,"cost":f
loat,"steps":int}
        rows.append({"task": t["id"], "pass_any": any(r["ok"] for
 r in runs),
                     "pass_all": all(r["ok"] for r in runs),
                     "cost": statistics.mean(r["cost"] for r in runs),
                     "steps": statistics.mean(r["steps"] for r in runs)})
    return rows
```

17.5 Project D — Domain-Specialised Small Model (Fine-Tune and Serve)

Conceptual Deep Dive

Goal: replace an expensive API on a high-volume narrow task (classification/extraction/summarisation) with a fine-tuned 3–8B model. Steps: log API outputs as teacher data, curate and review, LoRA/QLoRA SFT, evaluate against the API baseline on a frozen test set, quantise (AWQ/GGUF) and re-evaluate, serve on vLLM with structured output, load test, canary behind the gateway with fallback to the API, monitor drift. Decision criterion: cost per successful task at equal quality.

Cross-Connection Scenarios

Chapters 12, 13, 10, 15.

Production Anti-Patterns

- Evaluating the student only on teacher outputs (not human-verified labels).
- No fallback when the small model is low-confidence.
- Ignoring licence terms of the teacher outputs.

Minimal Working Code

```
def cost_per_success(cost_per_req: float, success_rate: float) -> f
loat:
    return cost_per_req / max(success_rate, 1e-9)

api = cost_per_success(0.004, 0.95)
small = cost_per_success(0.0004, 0.93)
print(f"API ${api:.5f} vs small ${small:.5f} per successful task; saving {1 - small/api:.0%}")
```

17.6 Capstone Checklist

Conceptual Deep Dive

A project is "production-ready" when it has: documented requirements and metrics; golden dataset with slices; CI with eval gates; tracing and dashboards; retries, timeouts, circuit breakers; guardrails and PII handling; least-privilege tools; cost budget and alerts; rollback plan; runbook and on-call owner; privacy/legal review; user feedback loop; and a post-launch review cadence.

Cross-Connection Scenarios

Chapters 10, 14, 15, 16.

Production Anti-Patterns

- Declaring done at the demo.
- No owner after launch.
- No budget alert.

Minimal Working Code

```
CHECKLIST = ["metrics", "golden_set", "ci_eva
l_gate", "tracing", "retries_timeouts", "guardrails",
            "pii_handling", "least_privilege", "cost_alerts", "rollba
ck", "runbook", "feedback_loop"]

def readiness(done: set[str]):
    missing = [c for c in CHECKLIST if c not in done]
    return {"score": 1 - len(missing) / len(CHECKLIST), "missing": missing}

print(readiness({"metrics", "tracing"}))
```

17.7 Project E — Document Intelligence Pipeline (Extraction at Scale)

Conceptual Deep Dive

Goal: turn invoices, contracts or forms into validated structured records. Pipeline: ingest files, classify document type, OCR/layout parse, route to a type-specific schema, extract with structured output (field plus source quote plus page), validate (types, arithmetic checks such as line items summing to total, cross-field rules), compute per-field confidence, and send low-confidence or failed-rule documents to a human review queue. Evaluate field-level precision/recall against a labelled set, and track straight-through-processing rate. Edge cases: multi-page tables, handwritten notes, rotated scans, multiple documents in one PDF, currency/locale formats, and duplicate submissions (idempotent on content hash). Cost levers: cheap model first, escalate on validation failure.

Cross-Connection Scenarios

Schema design (7.6), batch processing (4.8), human-in-the-loop (11.8), PII handling (14.3).

Production Anti-Patterns

- Trusting extracted totals without arithmetic validation.
- No human review path for low-confidence fields.
- Processing the same file twice due to missing dedupe.

Minimal Working Code

```
def validate_invoice(inv: dict, tol: float = 0.01) -> list[str]:
    errs = []
    s = sum(li["qty"] * li["unit_price"] for li in inv["line_items"])
    if abs(s - inv["subtotal"]) > tol:
        errs.append(f"line items sum {s:.2f} != subtotal {inv['subtotal']:.2f}")
    if abs(inv["subtotal"] + inv["tax"] - inv["total"]) > tol:
        errs.append("subtotal + tax != total")
    return errs

inv = {"line_items": [{"qty": 2, "unit_price": 10.0}], "subtotal"
: 20.0, "tax": 2.0, "total": 22.0}
print(validate_invoice(inv))
```

```
import hashlib

def route(doc_bytes: bytes, extracted: dict, errors: list[str], seen: set[str], min_conf=0.85):
    h = hashlib.sha256(doc_bytes).hexdigest()
    if h in seen:
        return "duplicate"
    seen.add(h)
    low = [f for f, v in extracted.items() if v["conf"] < min_conf]
    return "human_review" if errors or low else "auto_approve"

print(route(b"pdf", {"total": {"conf": 0.9}}, [], set()))
```

17.8 Project F — Voice and Realtime Assistant

Conceptual Deep Dive

Goal: a low-latency spoken assistant. Pipeline options: **cascaded** (streaming ASR to LLM to streaming TTS) or **native speech-to-speech** models. Latency budget for natural conversation is roughly 500–800 ms from end of user speech to first audio. Techniques: voice-activity detection and endpointing, streaming everything, sentence-level TTS chunking, interruption (barge-in) handling that cancels generation and playback, WebSocket or WebRTC transport, and short responses by prompt design. Tools via function calling must be fast or announce delays ("one moment"). Evaluate word error rate, time-to-first-audio, interruption success, task completion, and user-rated naturalness. Edge cases: background noise, accents, overlapping speech, echo from speakers, phone-line audio quality, and TTS mispronouncing names or numbers.

Cross-Connection Scenarios

Streaming and WebSockets (4.2), latency (2.4, 10.6), tools (7.4), guardrails on spoken output (14.4).

Production Anti-Patterns

- Waiting for the full LLM response before starting TTS.
- No barge-in, so users must wait for the bot to finish.
- Reading long lists and URLs aloud.

Minimal Working Code

```
import re

def sentence_chunks(token_stream):
    """Yield complete sentences as soon as they are available so TTS can start early."
;""
    buf = ""
    for tok in token_stream:
        buf += tok
        while (m := re.search(r"[.!?]\s", buf)):
            yield buf[:m.end()].strip()
            buf = buf[m.end():]
    if buf.strip():
        yield buf.strip()

print(list(sentence_chunks(iter(["Hello the", "re. How can ", "I help? Sure"
;])))))
```

```
import asyncio

async def speak_with_barge_in(tts_chunks, user_speaking: asyncio.Event, play):
    for chunk in tts_chunks:
        if user_speaking.is_set():
            return "interrupted"
        await play(chunk)
    return "completed"
```

17.9 Project G — Internal Evaluation and Observability Platform

Conceptual Deep Dive

Goal: shared infrastructure so every team ships measurable AI features. Components: trace ingestion (OpenTelemetry collector to a store), dataset management (versioned golden sets, labelling UI), eval runner (parallel, cached, budget-aware, pluggable metrics and judges), experiment comparison (paired stats, slice views), prompt/model registry, gating API for CI, and dashboards with cost and quality. Design decisions: multi-tenant access control, PII scrubbing at ingestion, sampling policy, retention, and an extensible metric interface. Success metrics: time from incident to regression test, share of changes shipped with an eval, and eval cost per run. Edge cases: judge-model version changes silently shifting scores (pin and recalibrate), large trace volumes, and reproducibility of old runs.

Cross-Connection Scenarios

Evaluation (10), observability (14.1), CI gates (15.2), prompt registry (15.6).

Production Anti-Patterns

- Each team building its own incompatible eval scripts.
- Dashboards with no link from a failing trace to a dataset case.
- Unpinned judge versions.

Minimal Working Code

```
from typing import Callable, Protocol

class Metric(Protocol):
    name: str
    def __call__(self, example: dict, output: str) -> float: ...

class ExactMatch:
    name = "exact_match"
    def __call__(self, example, output):
        return float(output.strip() == example["expected"].strip())

def run_eval(dataset: list[dict], system: Callable[[str], str], metrics: list[Metric]) -> dict:
    scores = {m.name: [] for m in metrics}
    for ex in dataset:
        out = system(ex["input"])
        for m in metrics:
            scores[m.name].append(m(ex, out))
    return {k: sum(v) / len(v) for k, v in scores.items()}

print(run_eval([{"input": "2+2", "expected": "4"}], lambda q: &qu
ot;4", [ExactMatch()]))
```

```
import hashlib
import json

def cached(fn):
    memo = {}
    def wrapper(example: dict, output: str):
        k = hashlib.sha256(json.dumps([example, output], sort_keys=True).encode()).hexdigest()
        if k not in memo:
            memo[k] = fn(example, output)
        return memo[k]
    return wrapper
```

17.10 Project H — Text-to-SQL Analytics Assistant

Conceptual Deep Dive

Goal: let business users ask questions in plain language and get correct, auditable numbers. Architecture: a **semantic layer** (curated views, metric definitions, synonyms, join paths) so the model targets business concepts rather than raw tables; **schema retrieval** (embed table/column descriptions, include only relevant ones plus sample values); generation with the schema and few-shot examples of correct queries; **static validation** (parse SQL, allow-list tables/columns, forbid DDL/DML, enforce LIMIT and time bounds, cost estimate via EXPLAIN); **read-only credentials** with row-level security tied to the user; execution with timeouts; result summarisation with the SQL shown for transparency; and a feedback loop that stores corrected queries as new few-shot examples and eval cases. Evaluate **execution accuracy** (result sets match gold) rather than string match. Edge cases: ambiguous metrics ("revenue" gross or net), fiscal calendars, time zones, join fan-out inflating sums, large result sets, and users asking questions the data cannot answer (abstain).

Cross-Connection Scenarios

Text-to-SQL (9.6), tool safety (7.7), eval of structured results (10.9), access control (4.7).

Production Anti-Patterns

- Pointing the model at the raw warehouse with broad credentials.
- Showing a number with no query or definition.
- Evaluating by SQL string similarity.

Minimal Working Code

```
import re

FORBIDDEN = re.compile(r"\b(insert|update|delete|drop|alter|create|grant|truncate|merge|copy|attach)
\b", re.I)

def validate_sql(sql: str, allowed_tables: set[str], max_limit: int = 1000) -> str:
    s = sql.strip().rstrip(";")
    if ";" in s or FORBIDDEN.search(s) or not re.match(r"(?is)^(with|select)\b", s):
        raise ValueError("only a single read-only SELECT is allowed")
    tables = {t.lower() for t in re.findall(r"(?i)\b(?:from|join)\s+([a-z_][\w.]*)", s)}
    ctes = {c.lower() for c in re.findall(r"(?i)\b([a-z_]\w*)\s+as\s*\(", s)}
    illegal = tables - ctes - allowed_tables
    if illegal:
        raise ValueError(f"tables not allowed: {sorted(illegal)}")
    m = re.search(r"(?i)\blimit\s+(\d+)", s)
    if not m:
        s += f" LIMIT {max_limit}"
    elif int(m.group(1)) > max_limit:
        raise ValueError("limit too large")
    return s

print(validate_sql("select region, sum(net_revenue) from sales_v group by region", {"sales
_v"}))
```

```
def execution_accuracy(cases, generate_sql, run_sql) -> float:
    ok = 0
    for c in cases:
        try:
            pred = sorted(map(tuple, run_sql(generate_sql(c["question"]))))
        except Exception:
            continue
        ok += pred == sorted(map(tuple, c["gold_rows"]))
    return ok / len(cases)

print(execution_accuracy([{"question": "q", "gold_rows": [(1,)]}], lambda q
: "select 1", lambda s: [(1,)]))
```

17.11 Project I — AI Code Review and Developer Productivity Assistant

Conceptual Deep Dive

Goal: automated review comments on pull requests that developers actually value. Pipeline: fetch the diff and changed-file context; run **deterministic tools first** (linters, type checkers, tests, secret scanners, dependency audits) and let the LLM focus on what tools miss (logic errors, missing edge cases, unclear naming, risky patterns, missing tests); retrieve repository conventions (style guide, similar code); generate comments with severity and a confidence score; **filter aggressively** (drop low-confidence and nitpick comments, dedupe against existing threads); post as a single review; learn from reactions (resolved/dismissed). Metrics: acceptance rate of comments, bugs caught before merge, reviewer time saved, false-positive rate. Safety: never run untrusted PR code with secrets, sandbox test execution, treat PR text as untrusted input (injection via code comments). Edge cases: huge diffs (chunk by file, prioritise risky files), generated files, force-pushes invalidating comments, and monorepo ownership rules.

Cross-Connection Scenarios

Coding agents (11.9), CI workflows (15.1), injection (6.5), cost governance (15.7).

Production Anti-Patterns

- Posting dozens of low-value comments that train developers to ignore the bot.
- Running PR code with repository secrets exposed.
- No measurement of whether comments were acted on.

Minimal Working Code

```
def parse_diff_files(diff: str) -> dict[str, list[str]]:
    files, cur = {}, None
    for line in diff.splitlines():
        if line.startswith("+++ b/"):
            cur = line[6:]
            files[cur] = []
        elif cur and line.startswith("+") and not line.startswith("+++"):
            files[cur].append(line[1:])
    return files

def prioritise(files: dict[str, list[str]], risky=("auth", "payment", "migration
", "security")) -> list[str]:
    def score(path):
        return (any(r in path.lower() for r in risky), len(files[path]))
    return sorted(files, key=score, reverse=True)

diff = "+++ b/app/auth/login.py\n+x = 1\n+++ b/README.md\n+docs\n"
print(prioritise(parse_diff_files(diff)))
```

```
def filter_comments(comments: list[dict], min_conf: float = 0.7, max_c
omments: int = 8, seen: set | None = None):
    seen = seen or set()
    keep = [c for c in comments
            if c["confidence"] >= min_conf and c["severity"] in {"bug",
"security", "risk"}
            and (c["file"], c["line"], c["kind"]) not in seen]
    keep.sort(key=lambda c: (c["severity"] != "security", -c["confidence"])
)
    return keep[:max_comments]

print(filter_comments([{"confidence": 0.9, "severity": "bug", "file&qu
ot;: "a.py", "line": 3, "kind": "npe"},
                       {"confidence": 0.5, "severity": "nit", "file&qu
ot;: "a.py", "line": 9, "kind": "style"}]))
```

17.12 Production Readiness Review Template and Cost Model

Conceptual Deep Dive

Before launch run a structured **readiness review** with sign-off from engineering, security, product and operations. Sections: *purpose and users*; *success metrics and ship criteria*; *architecture and data flows* (diagram incl. third parties); *evaluation evidence* (golden set size, slices, intervals, judge calibration); *safety* (red-team results, guardrails, false-positive rates); *privacy and compliance* (data classes, retention, DPIA if needed); *reliability* (SLOs, load test results, failure modes, fallbacks); *operations* (dashboards, alerts, runbooks, on-call owner, kill switch, rollback tested); *cost model* (per-request cost, monthly forecast at expected/10x traffic, budget alarms); *rollout plan* (shadow, canary, success/abort criteria); *post-launch plan* (feedback loop, review at 2 and 6 weeks). The **cost model** multiplies traffic by tokens by price, then adds retrieval, embedding, storage, GPU, observability and human review costs, and expresses everything per successful task. Edge cases: costs that scale with context growth over time, retries inflating spend, and seasonal traffic.

Cross-Connection Scenarios

Capstone checklist (17.6), cost (2.4, 13.5, 15.7), SLOs (13.7), governance (14.7).

Production Anti-Patterns

- Launch reviews held after the date is already announced.
- A cost model covering only LLM tokens.
- No abort criteria in the rollout plan.

Minimal Working Code

```
def monthly_cost_model(reqs_per_month: int, tin: int, tout: int, p_in:
 float, p_out: float,
                       retrieval_per_req: float = 0.0005, other_fixed: float = 2000.0,
                       retry_factor: float = 1.05, success_rate: float = 0.9) -> dict:
    llm = reqs_per_month * retry_factor * (tin / 1e6 * p_in + tout / 1e6 * p_out)
    retrieval = reqs_per_month * retrieval_per_req
    total = llm + retrieval + other_fixed
    return {"llm": round(llm), "retrieval": round(retrieval), "fixed": othe
r_fixed, "total": round(total),
            "per_request": total / reqs_per_month, "per_successful_task": total / (re
qs_per_month * success_rate)}

base = monthly_cost_model(1_000_000, 2500, 350, 0.5, 2.0)
tenx = monthly_cost_model(10_000_000, 2500, 350, 0.5, 2.0)
print(base["total"], tenx["total"], round(base["per_successful_task"], 4))
```

```
REVIEW_SECTIONS = ["purpose", "metrics", "arc
hitecture", "evals", "safety", "privacy",
                   "reliability", "operations", "cost", "rollout"
, "post_launch"]

def readiness_status(review: dict) -> dict:
    missing = [s for s in REVIEW_SECTIONS if not review.get(s, {}).get("approved_by")]
    return {"ready": not missing, "missing_signoffs": missing}

print(readiness_status({"purpose": {"approved_by": "pm"}, "cost":
 {"approved_by": "finance"}}))
```